

Taller de APIs (RESTful)

ComCom

Departamento de Computación
FCEyN UBA

Primer Cuatrimestre 2026



Introducción a las APIs

Interactuando con una API RESTful

Rutas / Routes / Endpoints

Middlewares

¿Qué es una API, y para qué sirve?

¿Cómo se comunican dos programas que no se conocen?

¹Visual Studio Code, Zed, los de JetBrains, Neovim, emacs, etc.

²Claude, Codex, Gemini, etc.

¿Qué es una API, y para qué sirve?

¿Cómo se comunican dos programas que no se conocen?

Las API (Application Programming Interface) son mecanismos que permiten a dos componentes de software comunicarse entre sí mientras ambos sistemas cumplan con un contrato/especificación.

¹Visual Studio Code, Zed, los de JetBrains, Neovim, emacs, etc.

²Claude, Codex, Gemini, etc.

¿Qué es una API, y para qué sirve?

¿Cómo se comunican dos programas que no se conocen?

Las API (Application Programming Interface) son mecanismos que permiten a dos componentes de software comunicarse entre sí mientras ambos sistemas cumplan con un contrato/especificación.

Ejemplos

- El lector de tarjetas cuando pagás en un local se comunica con la API de la pasarela de pagos.
- Tu IDE¹ favorito habla mediante una API con tu IA/Agente/LLM² de confianza.

¹Visual Studio Code, Zed, los de JetBrains, Neovim, emacs, etc.

²Claude, Codex, Gemini, etc.

Existen varios tipos de APIs, algunas son:

- RPC
- WebSocket
- **RESTful**

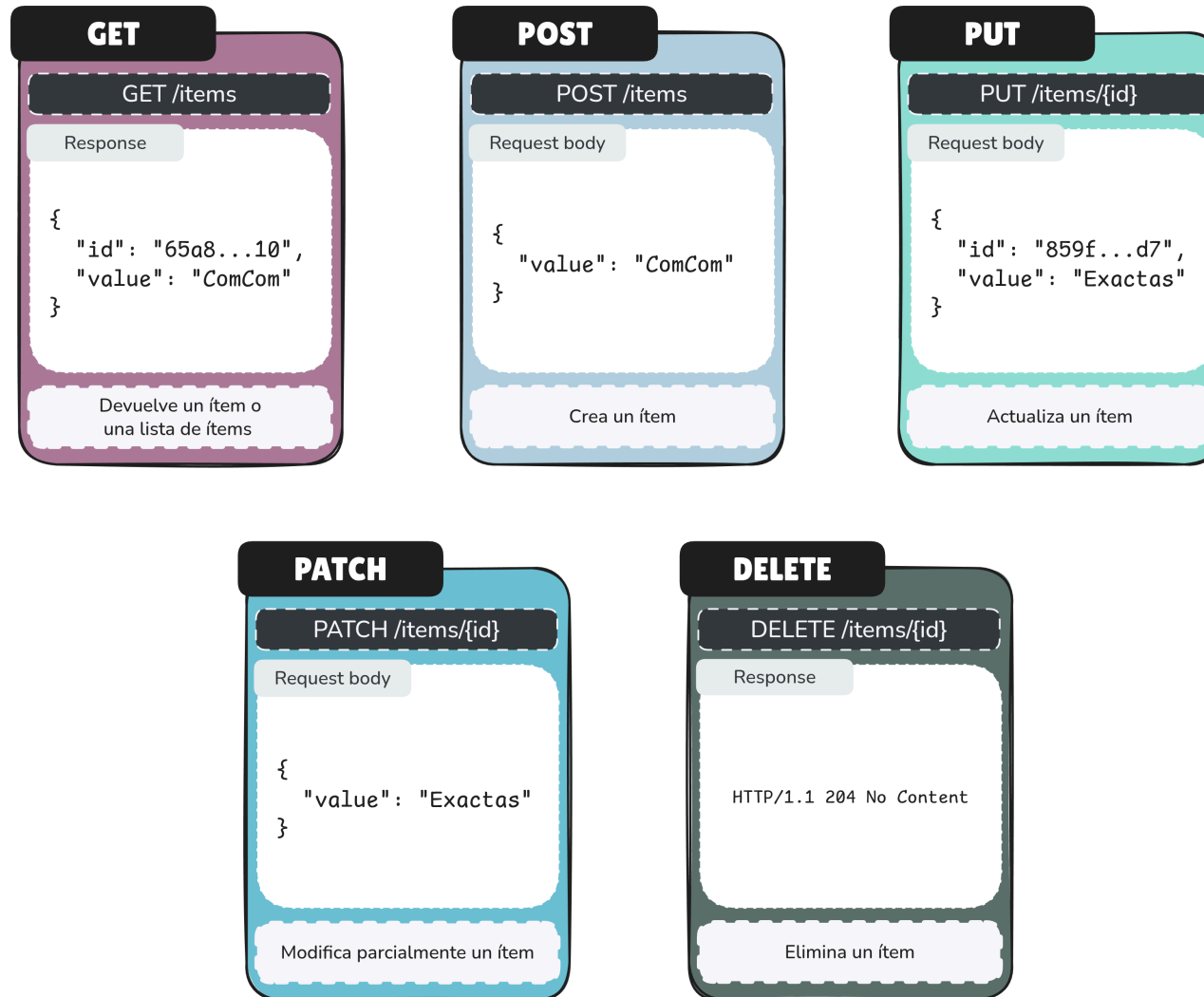
Siendo estas últimas las más famosas, y las que nos interesarán en el transcurso del taller.

Las APIs RESTful son un tipo de API que sigue los principios de diseño de **REST** (Representational State Transfer).

- Utilizan HTTP para realizar operaciones CRUD (Create, Read, Update, Delete) sobre recursos.

Métodos/verbos HTTP

Para entender como funcionan las APIs RESTful, es importante conocer los métodos HTTP más comunes:



¿Cómo devuelven la información?

Las APIs RESTful suelen devolver la información en formato JSON, aunque también pueden usar XML u otros formatos. A su vez, utilizan códigos de estado HTTP.

```
1 {  
2   "id": 1,  
3   "name": "Juan Pérez",  
4   "email": "jperez@dc.uba.ar",  
5   ...  
6 }
```

 JSON

HTTP/1.1 200 OK

Es importante remarcar que **no toda request devuelve contenido** en el cuerpo.

Por ejemplo, una request de eliminación exitosa del verbo DELETE suele devolver un código de estado 204 No Content, indicando que la operación fue exitosa pero no hay contenido para devolver.

Los códigos de estado HTTP son respuestas estándar que indican el resultado de la solicitud realizada a la API.

- 1xx: Informativos
- 2xx: Éxito
- 3xx: Redirección
- 4xx: Error del cliente
- 5xx: Error del servidor

HTTP status ranges in a nutshell:

1xx: hold on

2xx: here you go

3xx: go away

4xx: you fucked up

5xx: I fucked up

--via @abt_programming

Hay varias formas de interactuar con una API RESTful, algunas de las más comunes son:

- Desde la terminal usando herramientas como `curl` o `httpie`.
- Usando clientes HTTP como Postman o Insomnia.
- Usando la propia documentación de la API, si esta tiene una interfaz interactiva (como Swagger UI o Scalar).

```
λ ~/ curl http://localhost:3000/items/ \
  --header 'Accept: application/json, multipart/form-data, text/plain'
{"id":"dafe32f6-fcd3-45d3-9fec-ca7ae0fcdee9","value":"ComCom"}%
```

```
λ ~/ http GET http://localhost:3000/items/ \
  Accept:'application/json, multipart/form-data, text/plain'
HTTP/1.1 200 OK
Content-Length: 62
Content-Type: application/json; charset=utf-8
Date: Tue, 21 Apr 2026 17:49:55 GMT

{
  "id": "dafe32f6-fcd3-45d3-9fec-ca7ae0fcdee9",
  "value": "ComCom"
}
```

Cientes HTTP (p. ej.: Postman)

The screenshot displays the Postman application interface. On the left sidebar, under 'COLLECTIONS', there is a 'My Collection' entry. The main workspace shows a GET request to 'http://localhost:3000/items/'. The 'Params' tab is active, showing a table with two columns: 'Key' and 'Value', both containing the text 'Key' and 'Value' respectively. Below this, the 'Body' tab is selected, showing a JSON response:

```
{  "id": "8b952ff6-e6c6-4b62-9f4b-fc53bdb49059",  "value": "ComCom"}
```

. The status bar at the bottom indicates a '200 OK' response with a 72 ms duration and 184 B of data. The bottom of the interface includes a footer with 'Connect Git', 'Console', 'Terminal', 'Globals', 'Vault', and 'Tools'.

Overview GET http://localhost:3000/items/ No environment

Save Share

GET http://localhost:3000/items/ Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body Cookies Headers 3 Test Results

200 OK • 72 ms • 184 B

{ } JSON Preview Visualize

```
1 {
2   "id": "8b952ff6-e6c6-4b62-9f4b-fc53bdb49059",
3   "value": "ComCom"
4 }
```

Connect Git Console Terminal Globals Vault Tools

Documentación interactiva (p. ej.: Scalar)

The screenshot displays the Scalar API client interface. At the top, the browser address bar shows the URL `http://localhost:3000/docs#tag/items/GET/items/`. The main panel shows a GET request to `http://localhost:3000 /items/` with a status of `200 OK`. The response body is a JSON object: `{ "id": "d9d629c5-0a06-4e1d-aa86-4985d54ef7f5", "value": "ComCom" }`. The interface includes sections for Cookies, Headers, Query Parameters, and a Code Snippet. The bottom bar shows the Scalar logo and the text "Powered by Scalar".

Method	URL	Status	Size	Time
GET	http://localhost:3000 /items/	200 OK	62 B	9ms

Response Headers: 3

Response Body: application/json

```
1 {
2   "id": "d9d629c5-0a06-4e1d-aa86-4985d54ef7f5",
3   "value": "ComCom"
4 }
```

Code Snippet

```
3 --header 'Content-Type: application/json' \
4 --data '{
5   "value": "ComCom"
6 }'
```

Introducción a las APIs

Interactuando con una API RESTful

Rutas / Routes / Endpoints

Middlewares

Esta API RESTful expone:

- `/items`: permite solicitudes GET y POST.
- `/items/{id}`: permite solicitudes GET, PUT, PATCH y DELETE.

Consignas:

- Entren a **/docs** de la API, donde van a encontrar la documentación interactiva.
- Prueben los distintos verbos HTTP sobre las rutas indicadas y observen las respuestas que devuelve la API.¹

¹También pueden usar cURL

Introducción a las APIs

Interactuando con una API RESTful

Rutas / Routes / Endpoints

Middlewares

Una ruta (también conocida como endpoint) es una URL que representa un recurso o una colección de recursos en una API RESTful.

- Las rutas se utilizan para acceder a los recursos y realizar operaciones sobre ellos utilizando los verbos HTTP.

En el ejercicio de la lista anónima, las rutas expuestas por la API RESTful son:

En el ejercicio de la lista anónima, las rutas expuestas por la API RESTful son:

- `/items`: representa la colección de ítems y permite realizar operaciones como obtener un ítem aleatorio de la lista (GET) o crear un nuevo ítem (POST).

En el ejercicio de la lista anónima, las rutas expuestas por la API RESTful son:

- `/items`: representa la colección de ítems y permite realizar operaciones como obtener un ítem aleatorio de la lista (GET) o crear un nuevo ítem (POST).
- `/items/{id}`: representa un ítem específico identificado por su `id` y permite realizar operaciones como obtener los detalles de un ítem (GET), actualizar un ítem (PUT o PATCH) o eliminar un ítem (DELETE).

- Route parameters (p. ej., {id}): se utilizan para identificar recursos específicos.
 - `/users/:id`
- Query parameters: se pueden utilizar, por ejemplo, para filtrar, ordenar o paginar recursos.
 - `/users?age=30&sort=asc&page=2`
- Body: se utiliza para enviar datos al servidor en solicitudes POST, PUT o PATCH.
 - `{ "name": "Juan Pérez" }`
- Response: es la información que el servidor devuelve al cliente después de procesar una solicitud.
 - `{ "id": 1, "name": "Juan Pérez" }`
 - `HTTP/1.1 200 OK`

Introducción a las APIs

Interactuando con una API RESTful

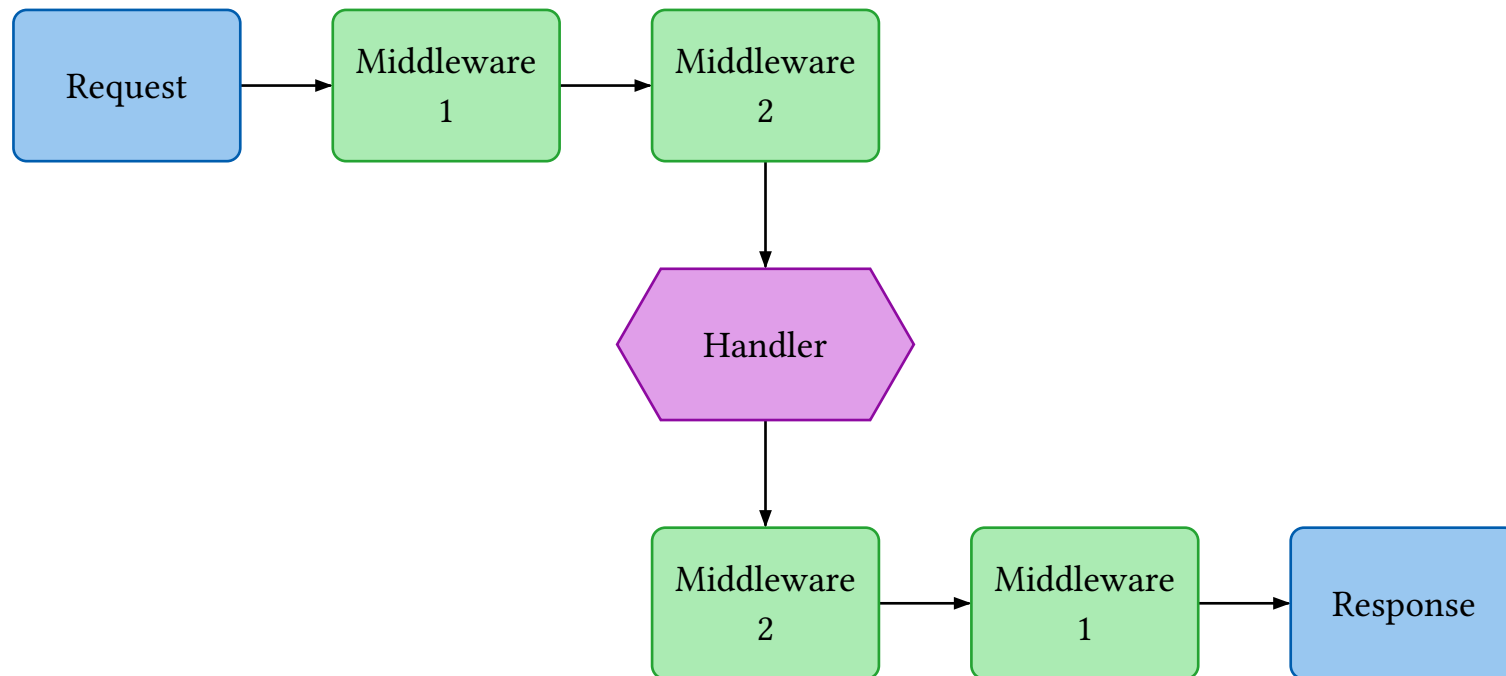
Rutas / Routes / Endpoints

Middlewares

¿Qué es un middleware?

Un middleware es una función que se ejecuta **antes** y/o **después** de que un handler procese una request.

Se puede pensar como una «capa» que intercepta las solicitudes y respuestas.



Los middlewares permiten agregar funcionalidad a la aplicación sin modificar los handlers individualmente. Por ejemplo:

- **Logging:** registrar cada solicitud que llega al servidor.
- **Autenticación:** verificar que el usuario tenga un token válido.
- **Autorización:** verificar que el usuario tenga permiso para acceder a la ruta.
- **Validación:** verificar que los datos de la solicitud sean correctos.

```
1  import { Elysia } from 'elysia'
2
3  export function middleware_ejemplo(app: Elysia) {
4    let middleware_app = app
5      .onBeforeHandle(({ request, set }) => {
6        // Se ejecuta antes de llegar al handler
7      })
8      .onAfterHandle(({ request, set }) => {
9        // Se ejecuta después de llegar al handler
10     });
11
12    return middleware_app;
13 }
```

TS TypeScript

¿Cómo se usa un middleware en Elysia?

Para usar un middleware, simplemente se agrega con `.use()`:

```
1  import { Elysia } from "elysia";  
2  import { logger } from "../middlewares/logger";  
3  import { authenticator } from "../middlewares/authenticator";  
4  
5  const app = new Elysia()  
6    .get("/sin-middleware", () => "No uso ningún middleware.")  
7    .use(logger)  
8    .get("/con-logger", () => "Uso logger.")  
9    .use(authenticator)  
10   .get("/secret", () => "Uso logger y luego authenticator.");
```

TS TypeScript

En el template tienen las rutas `/secret` y `/users` **sin protección...**

Consigna:

1. Abran los archivos `secret.routes.ts` y `users.routes.ts`.
2. Importen el middleware authenticator de `../middlewares/authenticator`.
3. Agreguen `.use(authenticator)` antes de la definición de la ruta.
4. Prueben acceder a `/secret` **sin** token: deberían recibir un `401 Unauthorized`.
5. Miren cómo se ve `/me` (`me.routes.ts`) como ejemplo de una ruta ya protegida.